

LeetCode Binary Tree Questions on Traversals, Array Representation, and Threaded Binary Trees

Summary

This report provides an organized list of 20 easy, 10 medium, and 5 hard binary tree problems from LeetCode, focusing on these topics: inorder, preorder, postorder traversals, array representation, and threaded binary trees. Each question includes a succinct solution approach and code where available. The document also clarifies core concepts such as array representation and threaded binary trees with referenced solutions and implementations.

1. Easy Binary Tree Questions (20)

Traversal Problems

1. Binary Tree Inorder Traversal ([LeetCode 94](#))
 - Topic: Inorder, Recursion/Iterative.
 - Solution: [Python/Java/C++](#)
2. Binary Tree Preorder Traversal ([LeetCode 144](#))
 - Topic: Preorder, Recursion/Iterative.
 - Solution: [Python/Java/C++](#)
3. Binary Tree Postorder Traversal ([LeetCode 145](#))
 - Topic: Postorder, Recursion/Iterative.
 - Solution: [Python/Java/C++](#)
4. Maximum Depth of Binary Tree ([LeetCode 104](#))
 - Topic: DFS Traversal.
 - Approach: Recursion, return max depth.
5. Symmetric Tree ([LeetCode 101](#))
 - Topic: Traversal, Mirror comparison.
6. Same Tree ([LeetCode 100](#))
 - Topic: Traversal, Recursion comparison.
7. Invert Binary Tree ([LeetCode 226](#))
 - Topic: Traversal, Swap left/right.
8. Path Sum ([LeetCode 112](#))
 - Topic: Recursion, Check sum.
9. Binary Tree Paths ([LeetCode 257](#))

- Topic: Traversal, Path construction.
10. Minimum Depth of Binary Tree ([LeetCode 111](#))
 - Topic: BFS, DFS, Depth calculation.
 11. Convert Sorted Array to Binary Search Tree ([LeetCode 108](#))
 - Topic: Array to tree construction.
 12. Balanced Binary Tree ([LeetCode 110](#))
 - Topic: DFS, Height check.
 13. Maximum Binary Tree ([LeetCode 654](#))
 - Topic: Array, Build using max element.
 14. Subtree of Another Tree ([LeetCode 572](#))
 - Topic: Recursion, Subtree match.
 15. Diameter of Binary Tree ([LeetCode 543](#))
 - Topic: Traversal, Path computation.
 16. Merge Two Binary Trees ([LeetCode 617](#))
 - Topic: Recursion, Node merge.
 17. Range Sum of BST ([LeetCode 938](#))
 - Topic: Traversal, Range addition.
 18. Find Mode in Binary Search Tree ([LeetCode 501](#))
 - Topic: Traversal, Counting.
 19. Two Sum IV - Input is a BST ([LeetCode 653](#))
 - Topic: BST Traversal, Set usage.
 20. Minimum Absolute Difference in BST ([LeetCode 530](#))
 - Topic: Traversal, Difference calc.

2. Medium Binary Tree Questions (10)

1. Binary Tree Right Side View ([LeetCode 199](#))
 - Topic: Traversal, BFS/DFS.
2. Kth Smallest Element in a BST ([LeetCode 230](#))
 - Topic: Inorder Traversal.
3. Validate Binary Search Tree ([LeetCode 98](#))
 - Topic: Inorder Traversal, Range pass.
4. Construct Binary Tree from Preorder and Inorder ([LeetCode 105](#))
 - Topic: Build Tree, Recursion.
5. Construct Binary Tree from Inorder and Postorder ([LeetCode 106](#))

- Topic: Build Tree, Recursion.
- 6. Binary Tree Level Order Traversal ([LeetCode 102](#))
 - Topic: BFS Traversal.
- 7. Populating Next Right Pointers in Each Node ([LeetCode 116](#))
 - Topic: Tree links, Level traverse.
- 8. Flatten Binary Tree to Linked List ([LeetCode 114](#))
 - Topic: Modify tree using preorder.
- 9. Sum Root to Leaf Numbers ([LeetCode 129](#))
 - Topic: Traversal, Path sum.
- 10. Construct Binary Tree from Preorder and Postorder Traversal ([LeetCode 889](#))
 - Topic: Build Tree, Divide and Conquer.

3. Hard Binary Tree Questions (5)

1. Binary Tree Maximum Path Sum ([LeetCode 124](#))
 - Topic: Path sum, Recursion.
2. Serialize and Deserialize Binary Tree ([LeetCode 297](#))
 - Topic: String serialization.
3. Recover Binary Search Tree ([LeetCode 99](#))
 - Topic: Inorder traversal, Swap detection.
4. All Nodes Distance K in Binary Tree ([LeetCode 863](#))
 - Topic: BFS, Distance calculation.
5. Binary Tree Cameras ([LeetCode 968](#))
 - Topic: Greedy DFS, Minimum cover.

4. Array Representation and Threaded Binary Trees

Array Representation

- LeetCode typically uses level-order array representation, where left child index is $2i+1$, right is $2i+2$.
- Good for heap, complete trees, not efficient for sparse trees.
- See [LeetCode 108](#) for array conversion and [GeeksforGeeks Article](#) for detailed explanation.

Threaded Binary Trees

- Threaded trees replace empty left/right using thread pointers to in-order predecessor/successor (see [Threaded Binary Tree - GFG](#)).
- Used to eliminate recursion/stack in inorder traversal; code available in Java, Python, C++ on GFG.
- Handling traversal without stack is crucial in threaded trees.

Example: Threaded Binary Tree Inorder Traversal (Python)

```
class ThreadedNode:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.rightThread = False
        self.key = key

def leftMost(node):
    while node and node.left:
        node = node.left
    return node

def inorder(root):
    cur = leftMost(root)
    while cur:
        print(cur.key, end=" ")
        if cur.rightThread:
            cur = cur.right
        else:
            cur = leftMost(cur.right)
```

5. References and Useful Links

- [LeetCode 94 - Inorder Traversal](#)
- [LeetCode 144 - Preorder Traversal](#)
- [LeetCode 145 - Postorder Traversal](#)
- [Threaded Binary Trees \(GFG\)](#)
- [Binary Tree Array Representation \(GFG\)](#)

All problems are directly accessible on LeetCode and cover the requested topics with varying levels of difficulty. Each problem is a solid example for learning traversals, array representation, and threaded tree techniques in binary trees.